Web Application Security Assessment

Vulnerability Findings & Evidence

Target: https://target.example.test
Report: Selected findings from the original assessment

About this report

This report contains five findings from my web application security testing. I have kept the test flow, observations, impact and recommended fixes in straightforward language. Target identifiers and personal details have been masked in the text and screenshots.

Screenshots are from the original report. Only detected sensitive areas—such as host details, account identifiers, tokens and certificate personal details—have been obscured; the surrounding testing evidence is retained.

Findings

AUTHZ-001 — Broken Function Level Authorization (BFLA) — Participant Access to Administrative Operations

INFO-03-001 — Production JavaScript Source Maps Publicly Accessible

CONF-003 — Unauthenticated Spring Boot Actuator Endpoints

AUTHZ-002 — Protected Email Address and Mobile Number Can Be Changed Through the Profile API

AUTHZ-002 — IDOR / Broken Object Level Authorization — Access to Other Participants’ Certificates

AUTHZ-001 — Broken Function Level Authorization (BFLA) — Participant Access to Administrative Operations

Severity: Critical

What I found

I found that changing the role value in the browser cookie made the administrative dashboard appear. More importantly, I tested an administrative Target Audience Management action and the server accepted it while I was still using a participant JWT. This showed that the issue was not only a front-end display problem: the backend did not consistently check whether the logged-in user had the required administrator role.

How I tested it

1. I logged in with a normal participant account and checked the role cookie in the browser.

2. I changed the cookie value from PARTICIPANT to ADMIN and refreshed the page. The admin dashboard and administrative menu items appeared.

3. I then opened Target Audience Category Management and tested an administrative create/edit action.

4. The request succeeded with the participant JWT. This confirmed that the backend accepted a privileged operation without the required role check.

Why it matters

A normal participant could perform administrative actions and change privileged application data. If similar checks are missing from other admin APIs, the same weakness could affect more administrative functions and the integrity of business records.

Why it happened

The browser cookie was trusted for rendering the user's role, and the backend did not consistently enforce role-based access on the administrative API endpoints.

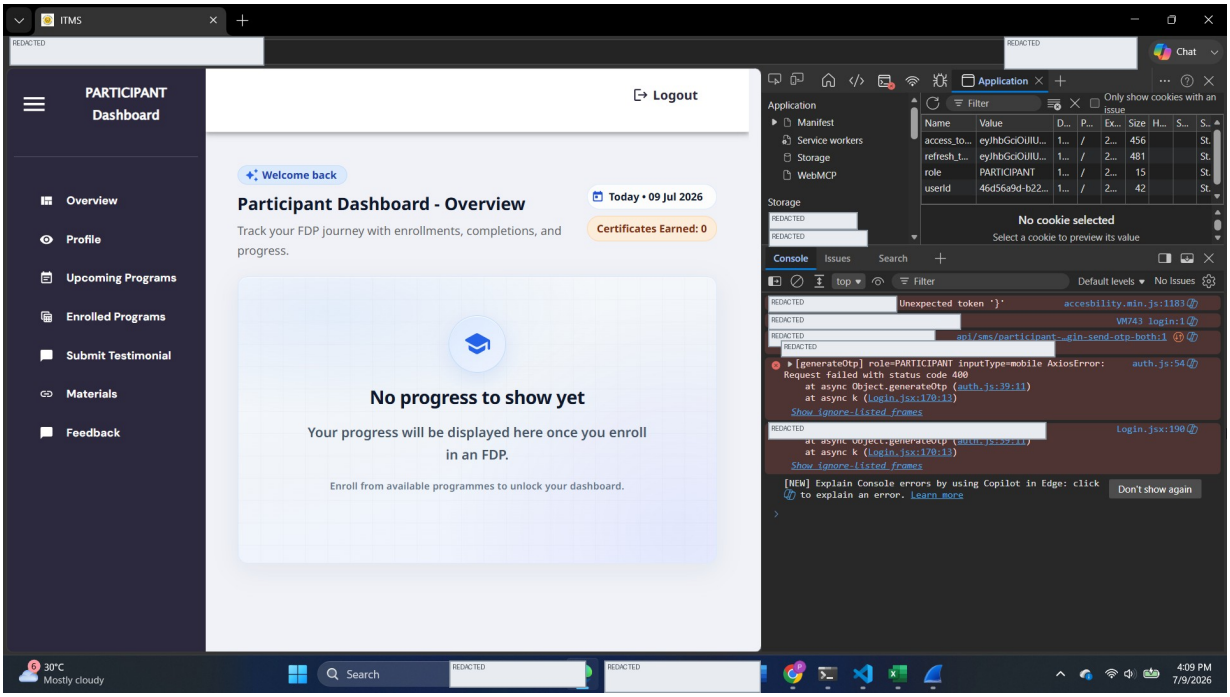
Recommended fix

Enforce authorization on the server for every administrative endpoint. Read the role from the validated server-side authentication context, not from a browser-controlled cookie. A participant request to an admin-only endpoint should be rejected even if the UI displays an admin page.

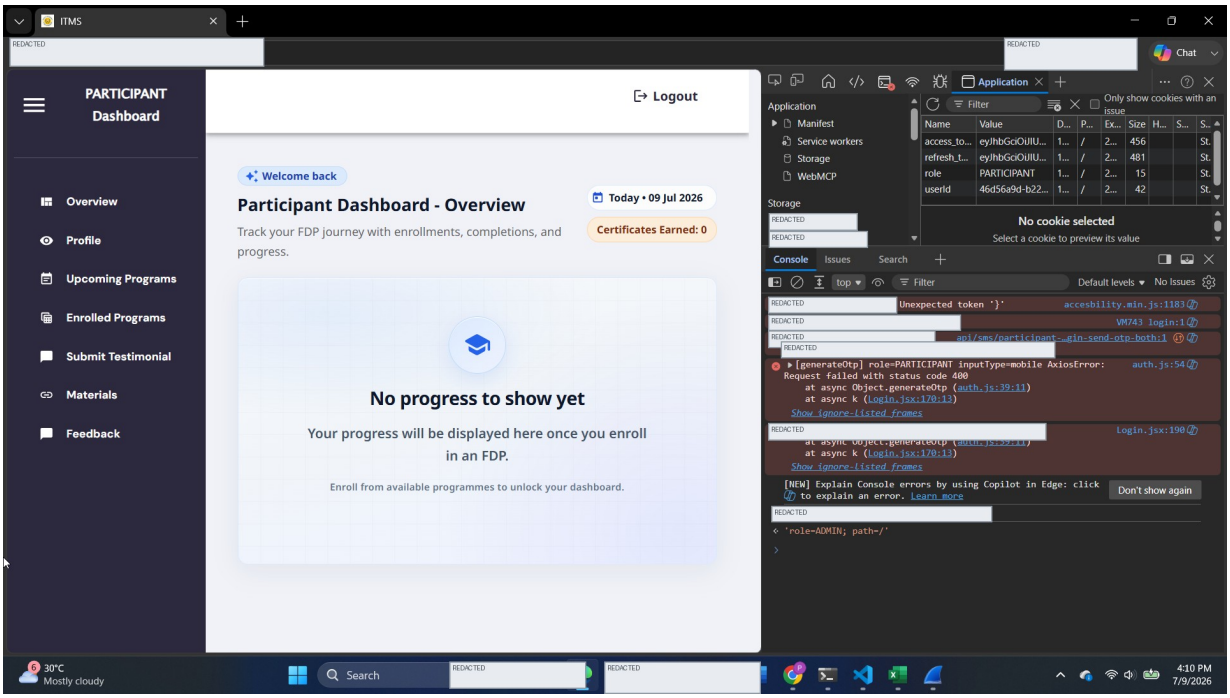
References

OWASP API Security Top 10 2023 — API5: Broken Function Level Authorization; CWE-285; WSTG-ATHZ-02 / WSTG-ATHZ-03

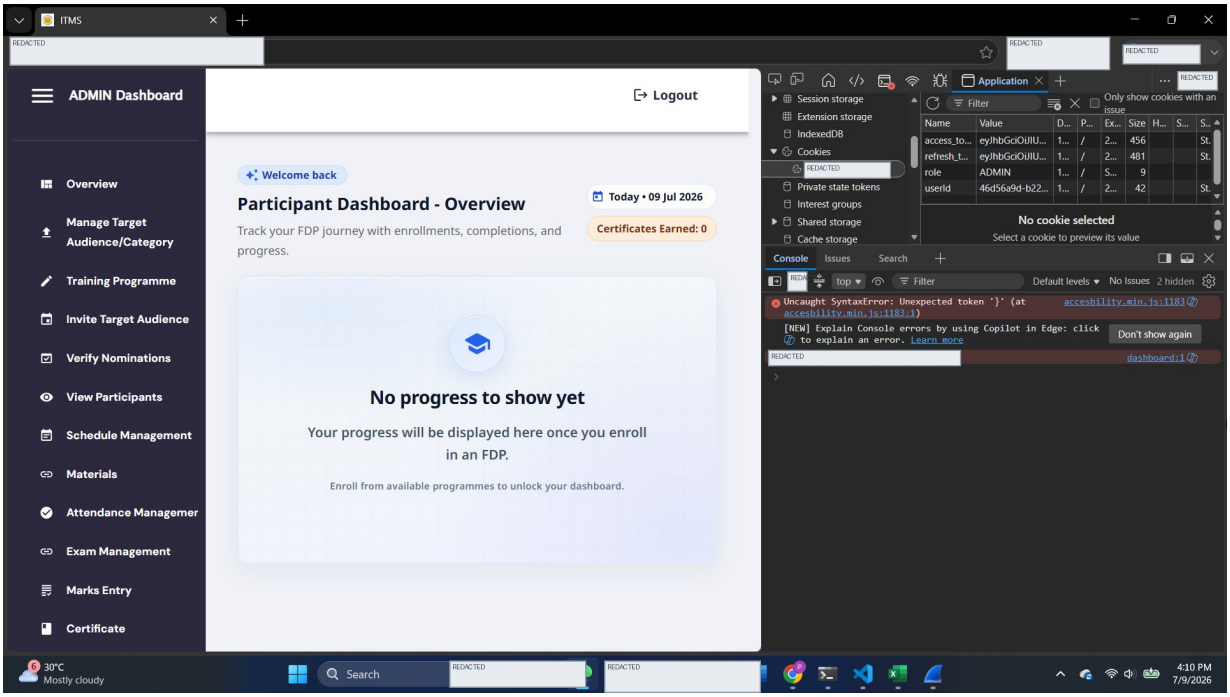
Evidence screenshots



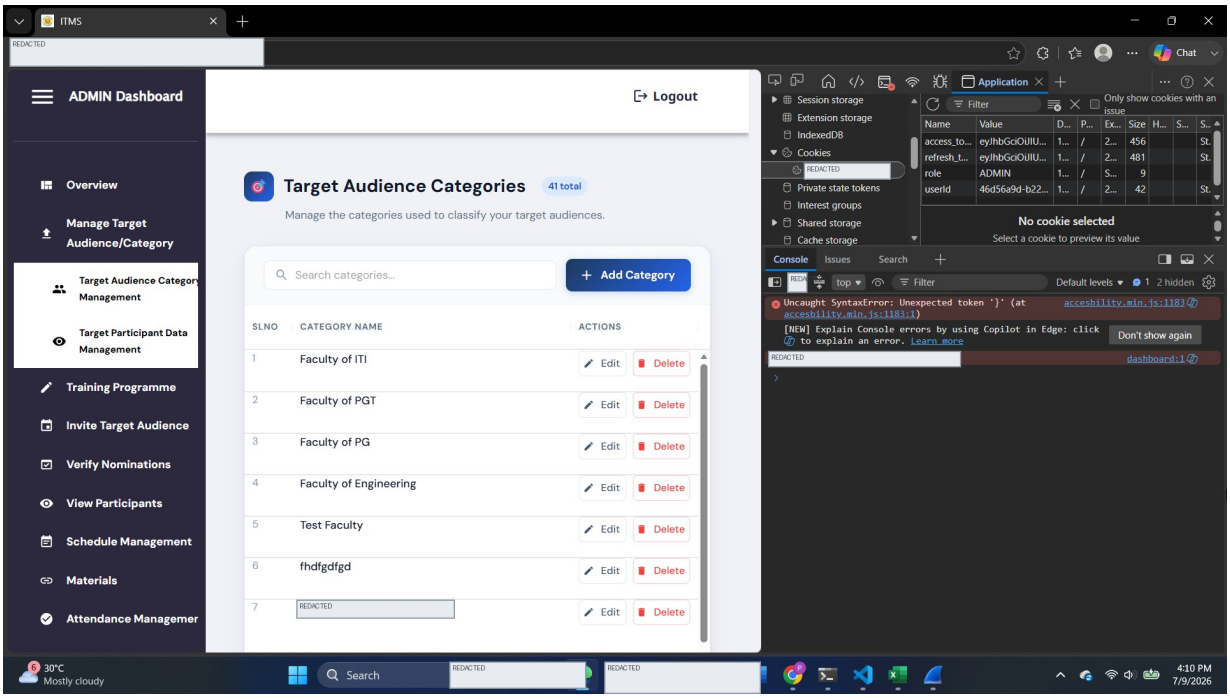
Original testing screenshot — sensitive details masked.



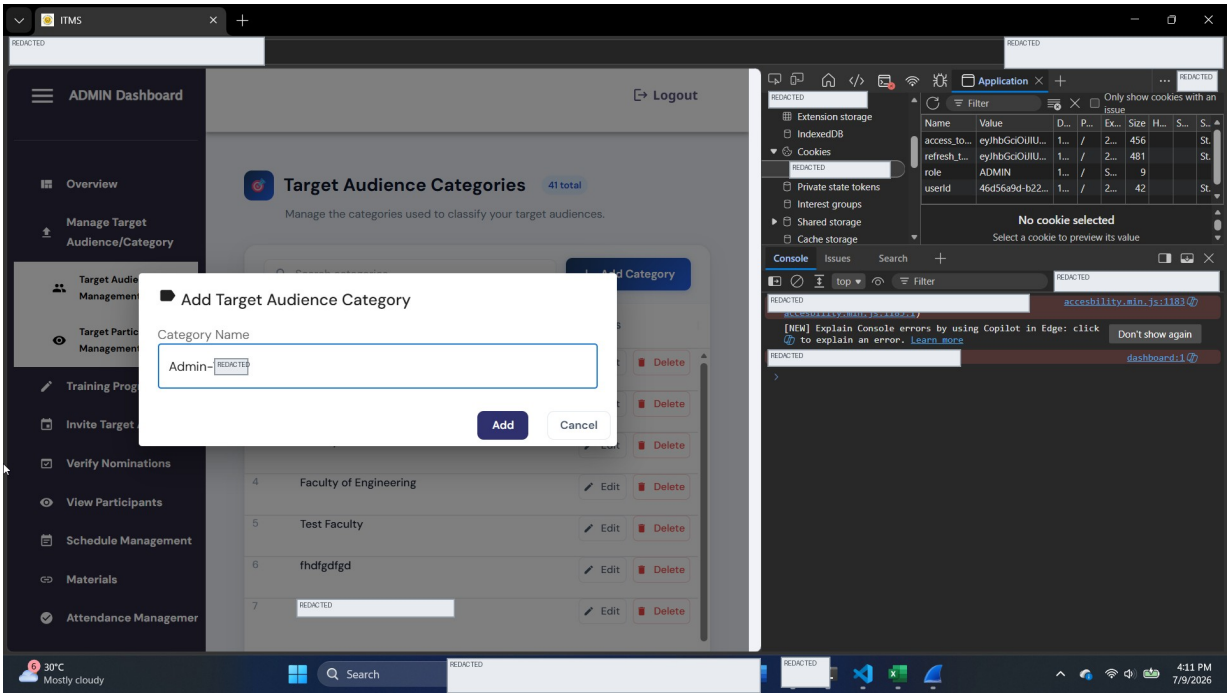
Original testing screenshot — sensitive details masked.



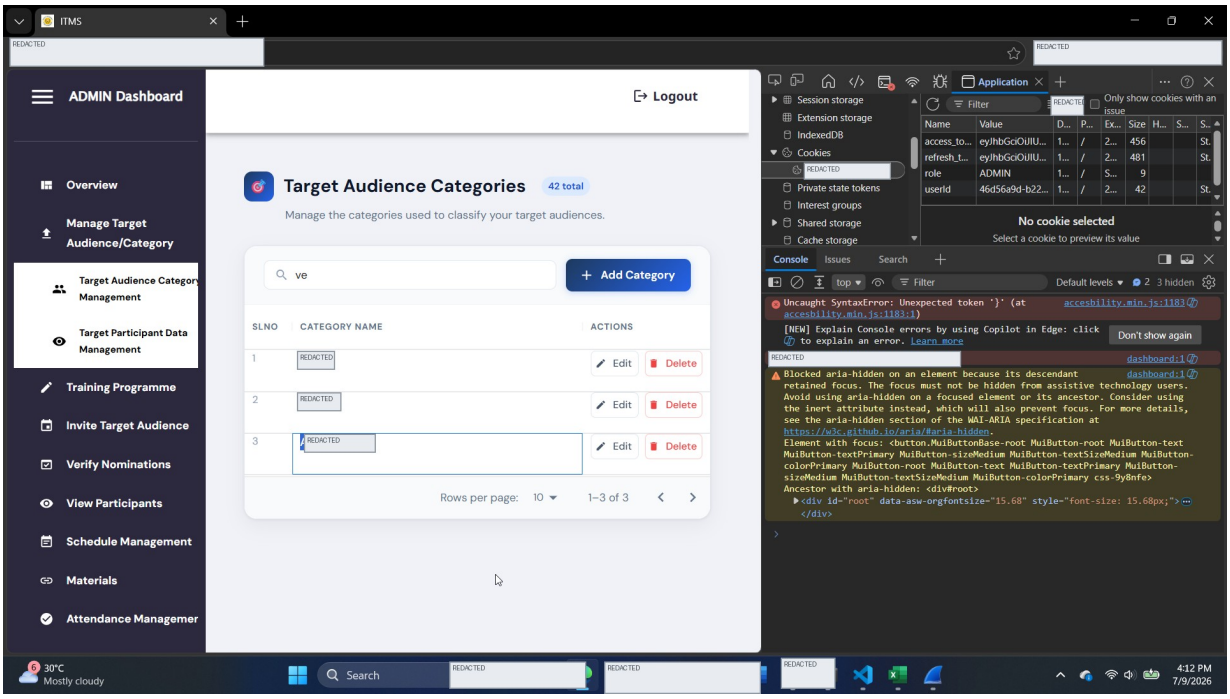
Original testing screenshot — sensitive details masked.



Original testing screenshot — sensitive details masked.



Original testing screenshot — sensitive details masked.



Original testing screenshot — sensitive details masked.

INFO-03-001 — Production JavaScript Source Maps Publicly Accessible

Severity: Medium

What I found

I checked the production JavaScript bundle and found that its matching source map was also publicly available. I downloaded the map and reconstructed approximately 172 original React source files. The recovered files included authentication components, API modules, routes, dashboard components and administrative functionality.

How I tested it

1. I opened the application and noted the JavaScript bundle loaded by the browser.

2. I requested the matching .map file from the same static asset location.

3. The server returned the source map successfully with HTTP 200.

4. I used a source-map reconstruction tool and recovered approximately 172 original React source files, including authentication-related components.

Why it matters

An attacker could inspect client-side application structure, API paths, routing and authentication-related logic. In this assessment, the recovered source also helped me understand how the client handled role information and identify further areas to test.

Why it happened

Source maps were generated for the production build and deployed alongside the public JavaScript bundle.

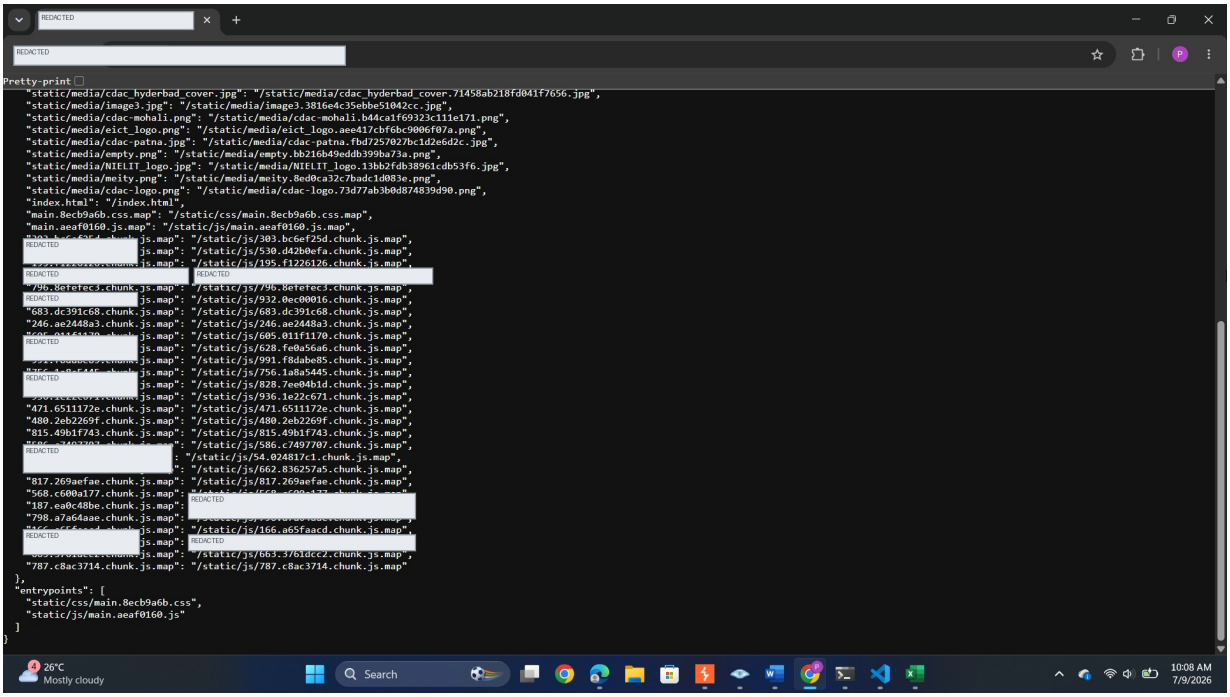
Recommended fix

Do not publish source map files in production unless there is a clear operational need. Remove existing public .map files and configure the build or web server to prevent public access to them.

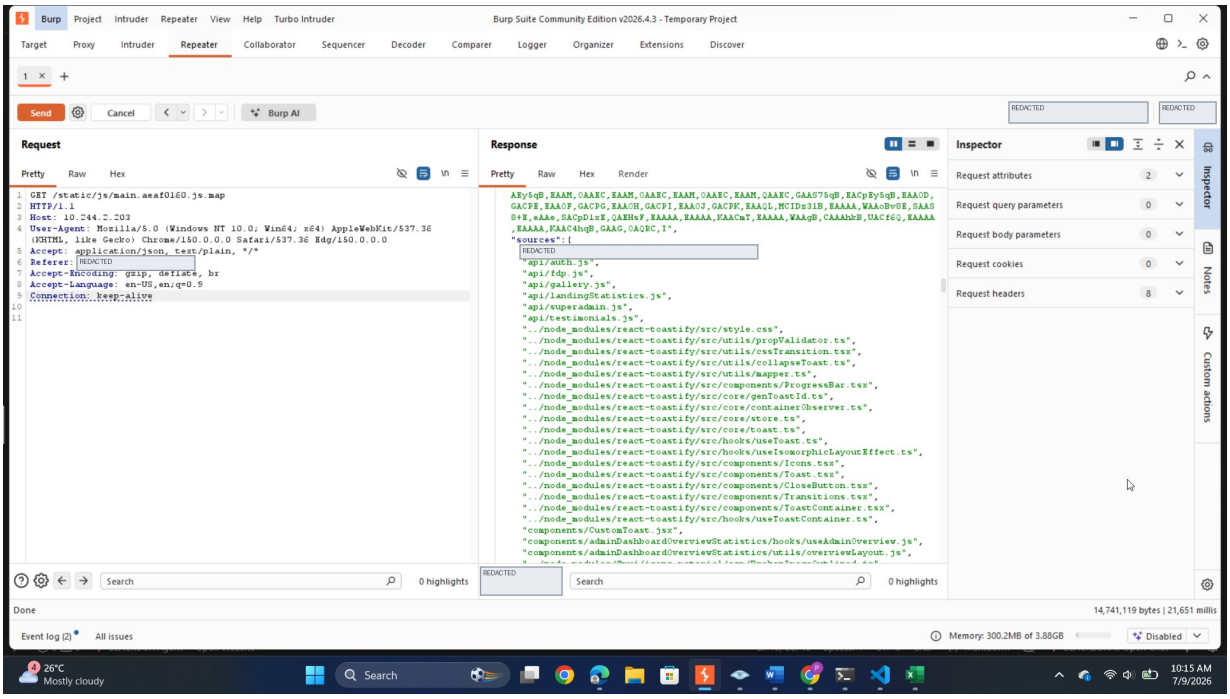
References

CWE-200; WSTG-CONF-05; React production build/source map configuration

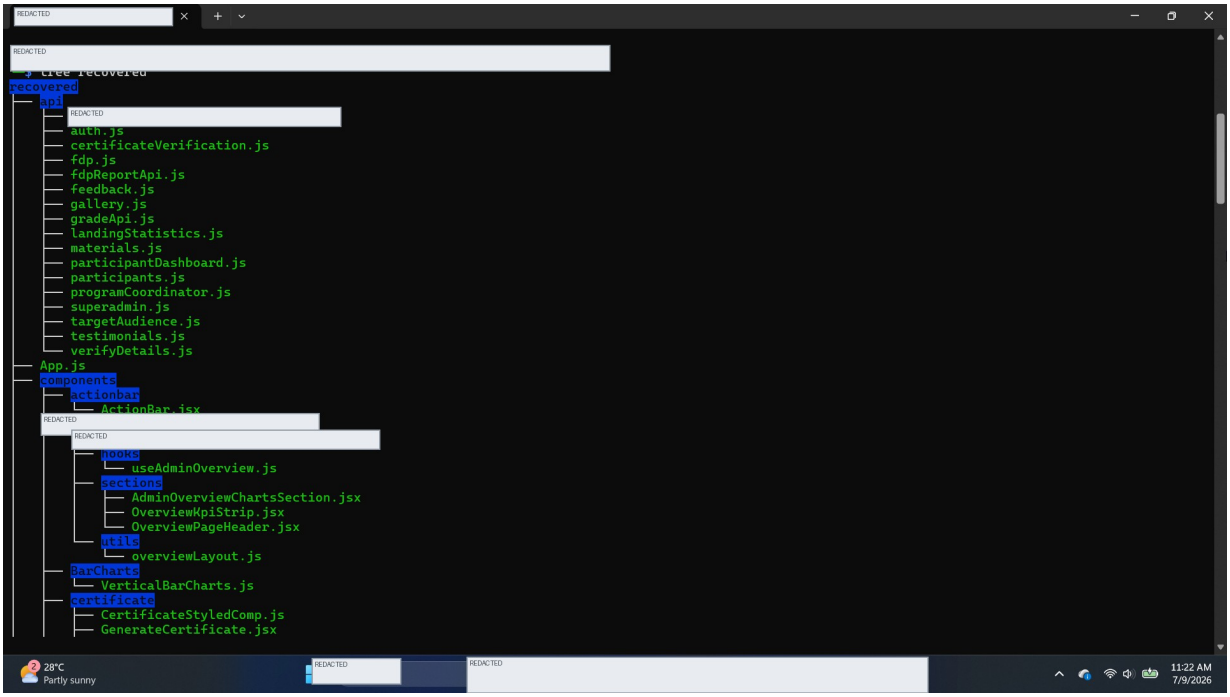
Evidence screenshots



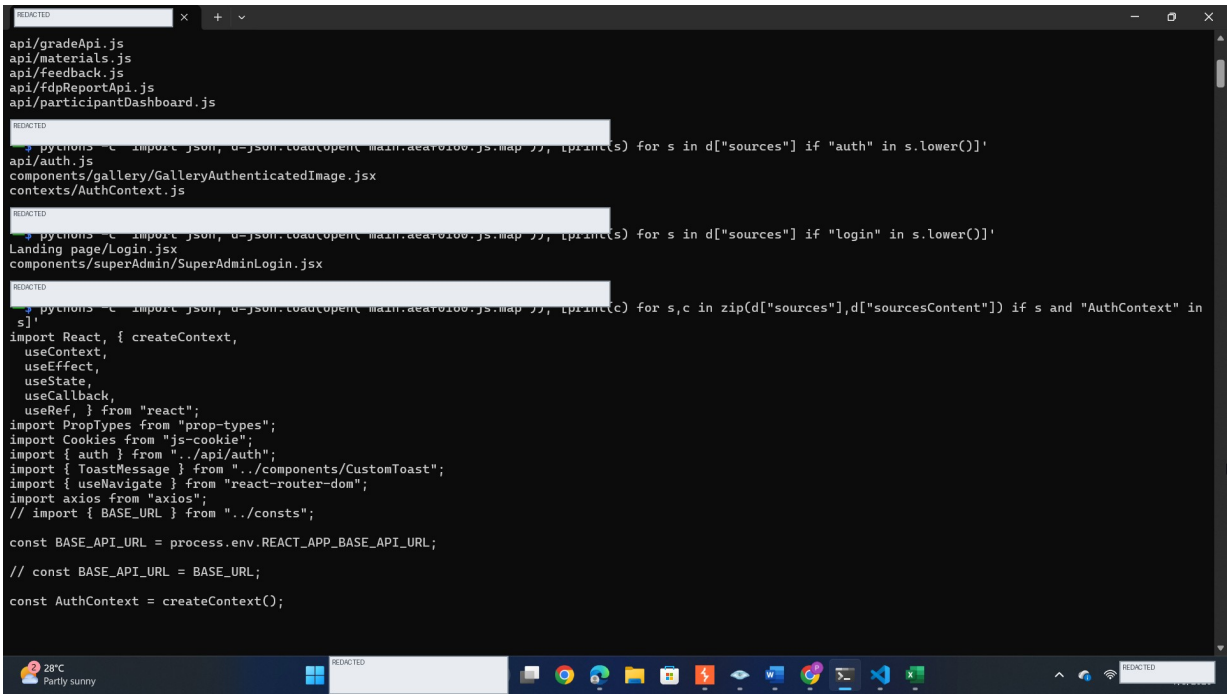
Original testing screenshot — sensitive details masked.



Original testing screenshot — sensitive details masked.



Original testing screenshot — sensitive details masked.



Original testing screenshot — sensitive details masked.

CONF-003 — Unauthenticated Spring Boot Actuator Endpoints

Severity: Medium

What I found

I tested the Spring Boot Actuator management endpoints and found that several of them responded without authentication. The responses exposed runtime metrics and internal application details, including JVM and memory information, thread statistics, microservice names, gateway routes and API paths.

How I tested it

1. I sent a GET request to /actuator/health and received a response without logging in.

2. I requested /actuator/metrics and observed JVM, CPU, memory, thread and gateway-related metrics.

3. I requested /actuator/ratelimiters and found internal service names, gateway routes and API path information in the response.

4. The tested endpoints returned HTTP 200 without requiring an authenticated or privileged session.

Why it matters

This information gives an attacker a clearer picture of the application's technology stack and internal structure. It can help identify additional endpoints and services to investigate. I did not demonstrate code execution through these endpoints.

Why it happened

Management endpoints were exposed to users who were not authenticated, rather than being restricted to administrators or trusted monitoring systems.

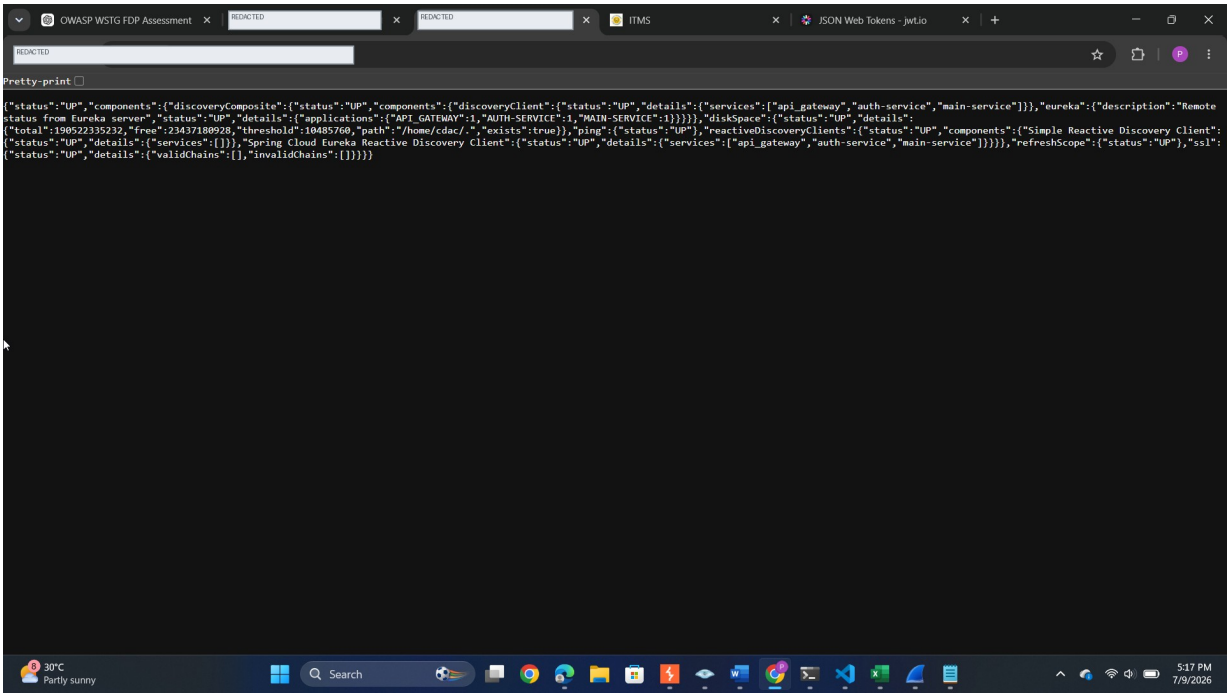
Recommended fix

Require authentication and authorization for management endpoints. Disable endpoints that are not needed in production and expose only the minimum information required for monitoring.

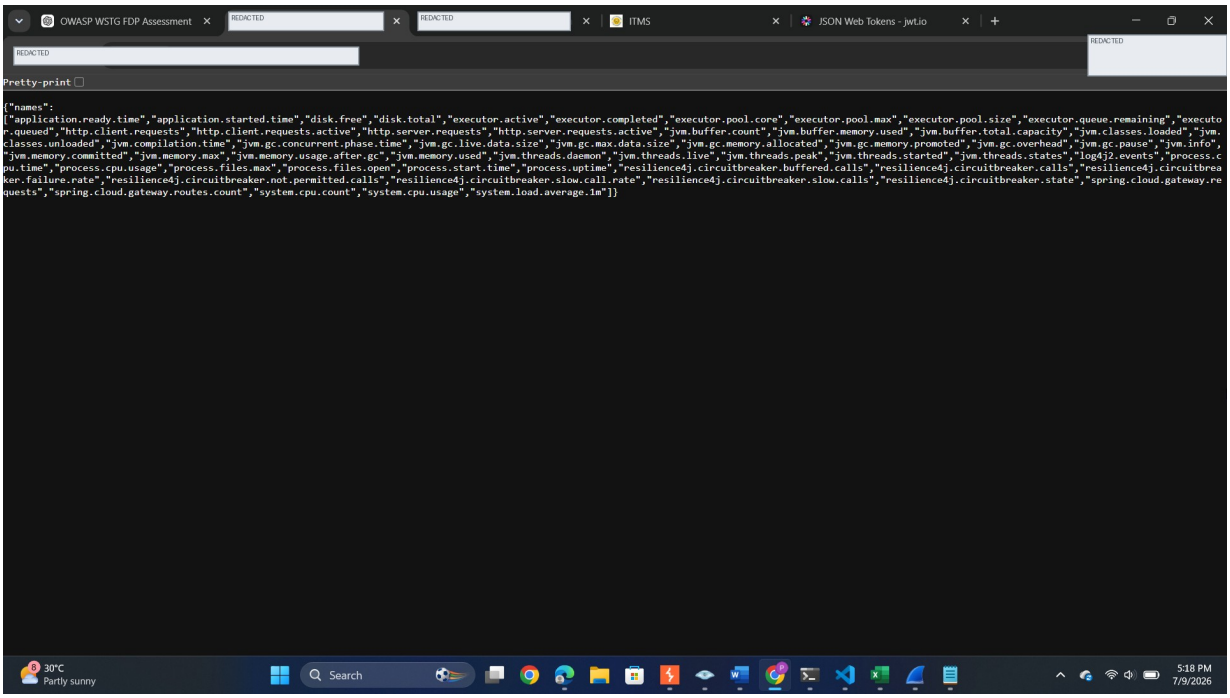
References

CWE-200; Spring Boot Actuator security documentation; WSTG-CONF

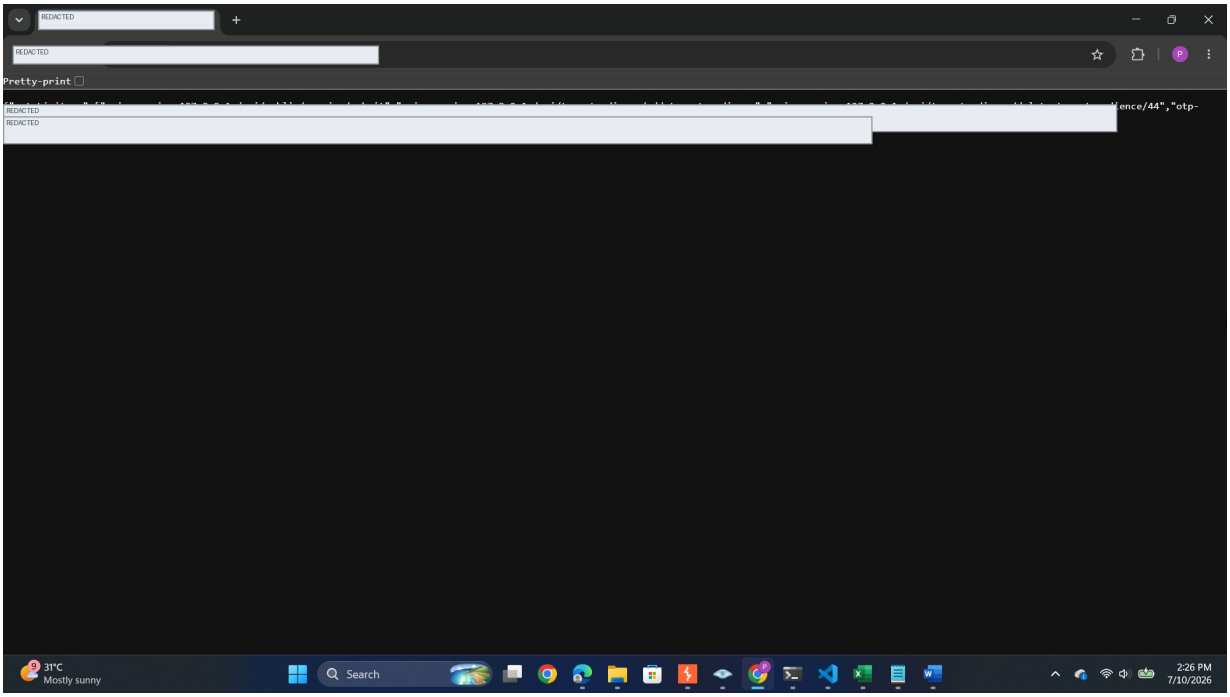
Evidence screenshots



Original testing screenshot — sensitive details masked.



Original testing screenshot — sensitive details masked.



Original testing screenshot — sensitive details masked.

AUTHZ-002 — Protected Email Address and Mobile Number Can Be Changed Through the Profile API

Severity: Medium

What I found

On the participant profile page, the email address and mobile number fields appeared disabled. I intercepted the profile update request and changed those values in the request body. The server accepted the modified request and updated the account details. This showed that the restriction was applied in the interface but was not properly enforced by the backend.

How I tested it

1. I logged in as a participant and opened the Profile page. The email address and mobile number fields were displayed as non-editable.

2. I intercepted the PUT /api/participant/update-profile request in Burp Suite.

3. I changed the email and mobile number values in the request and forwarded it.

4. The server accepted the request and the profile showed the updated values.

Why it matters

A logged-in participant could change account identifiers that the interface was supposed to protect. If these identifiers are used for login, verification or account recovery, changing them without a dedicated verification process could affect account integrity and recovery.

Why it happened

The application relied on disabled form fields in the browser. The profile update API still accepted changes to email and mobile number.

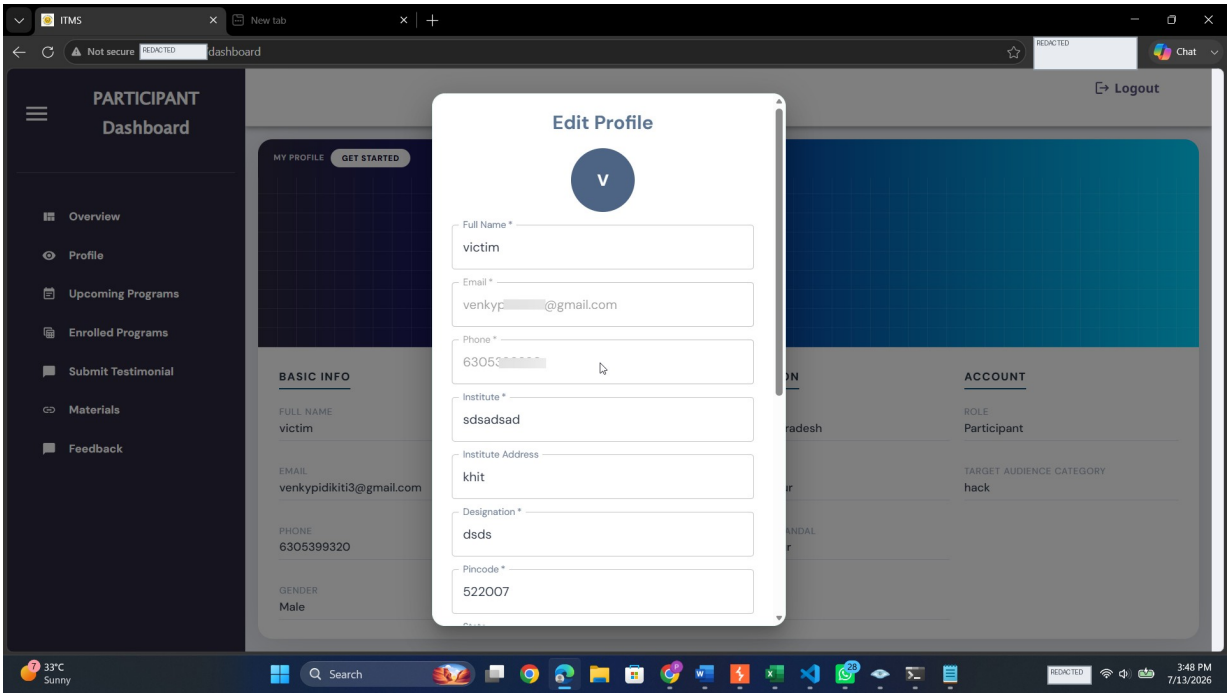
Recommended fix

Allow the general profile endpoint to update only approved fields. Reject changes to email and mobile number through this endpoint, and use a separate verified change process for those identifiers.

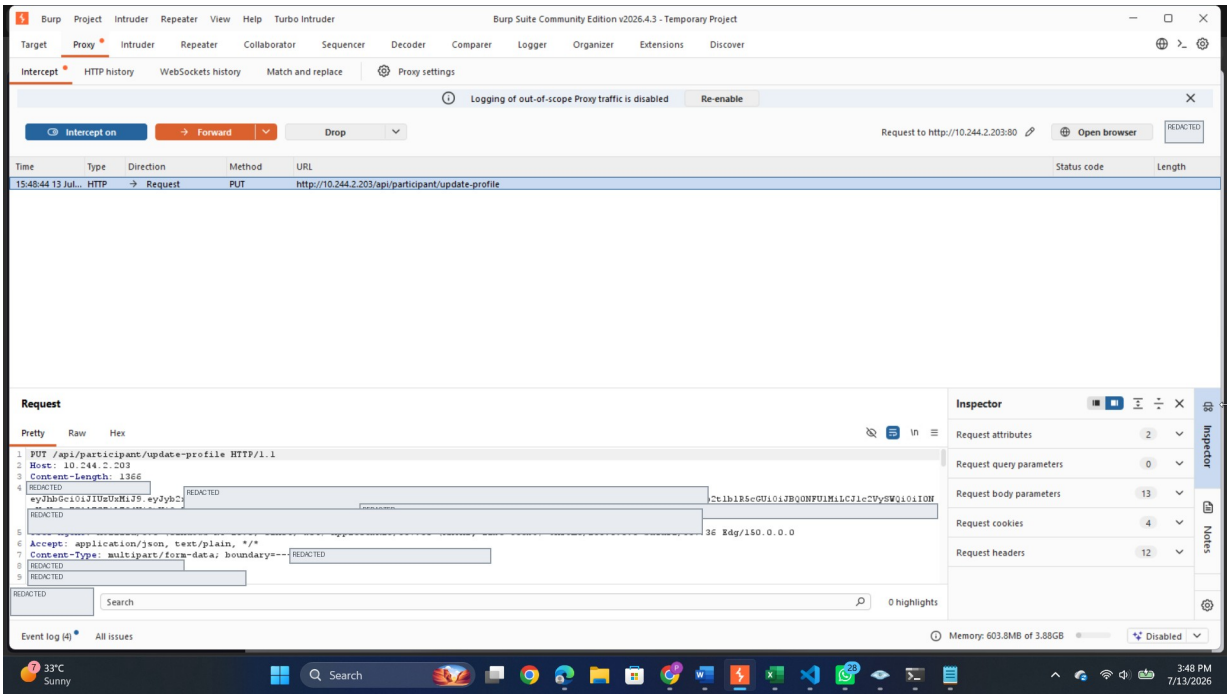
References

CWE-285; CWE-602; OWASP A01 Broken Access Control; WSTG-ATHZ-02

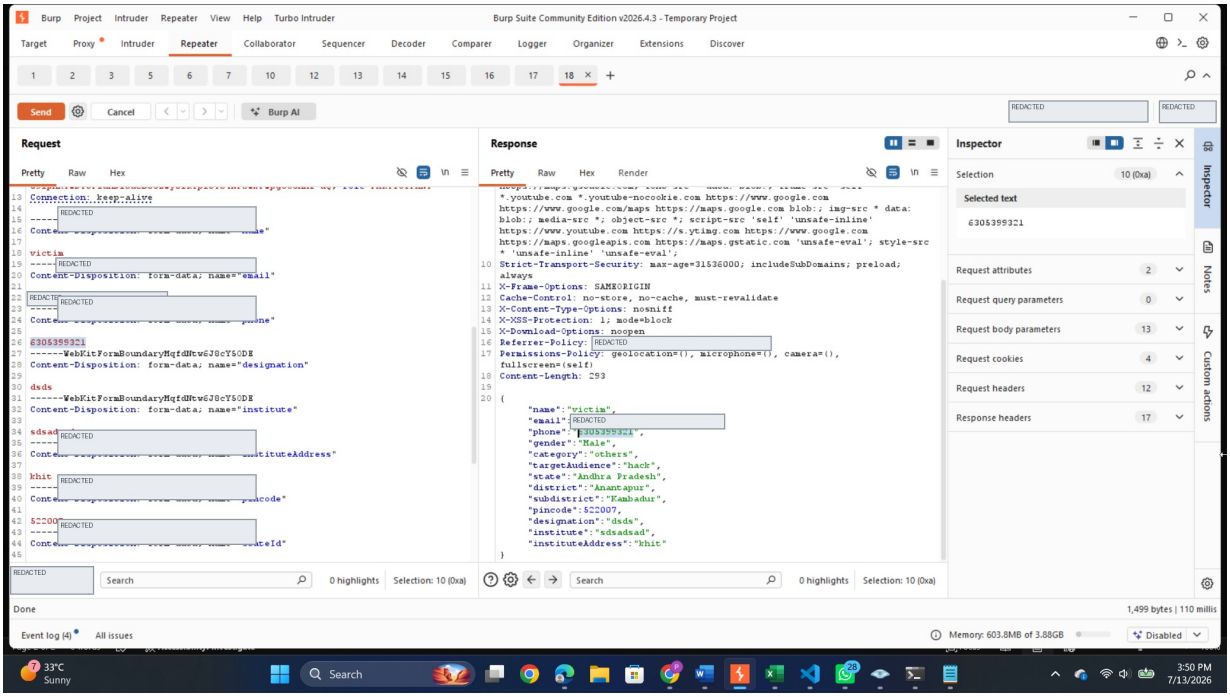
Evidence screenshots



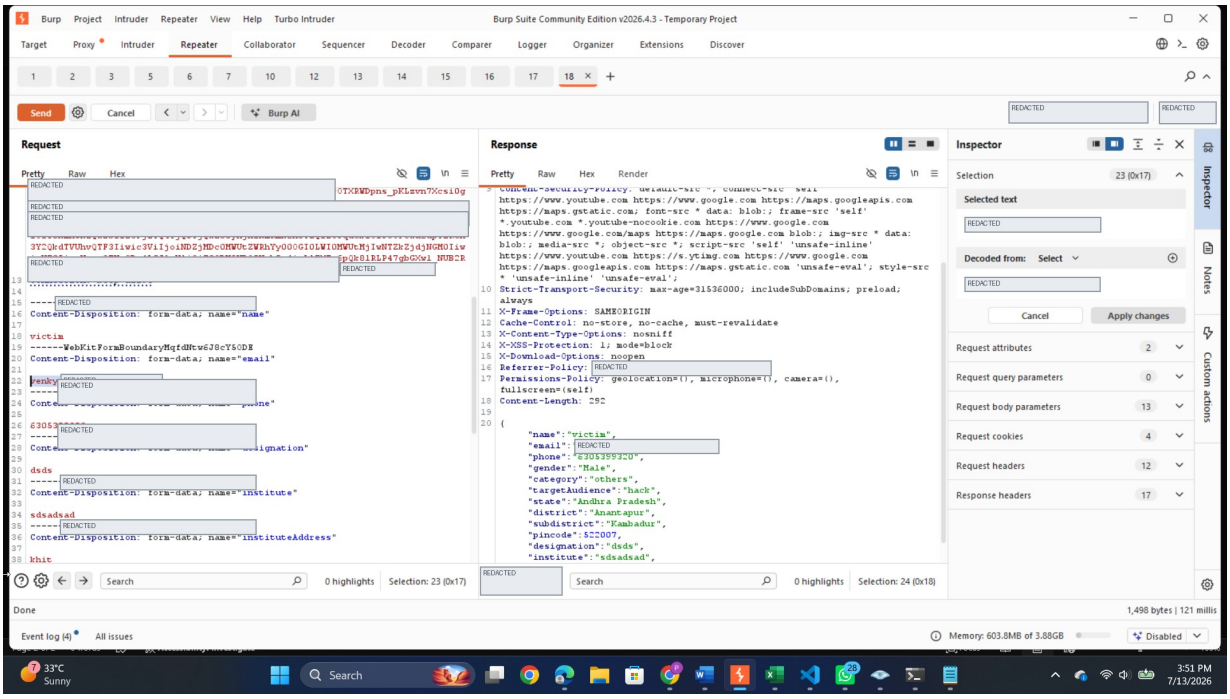
Original testing screenshot — sensitive details masked.



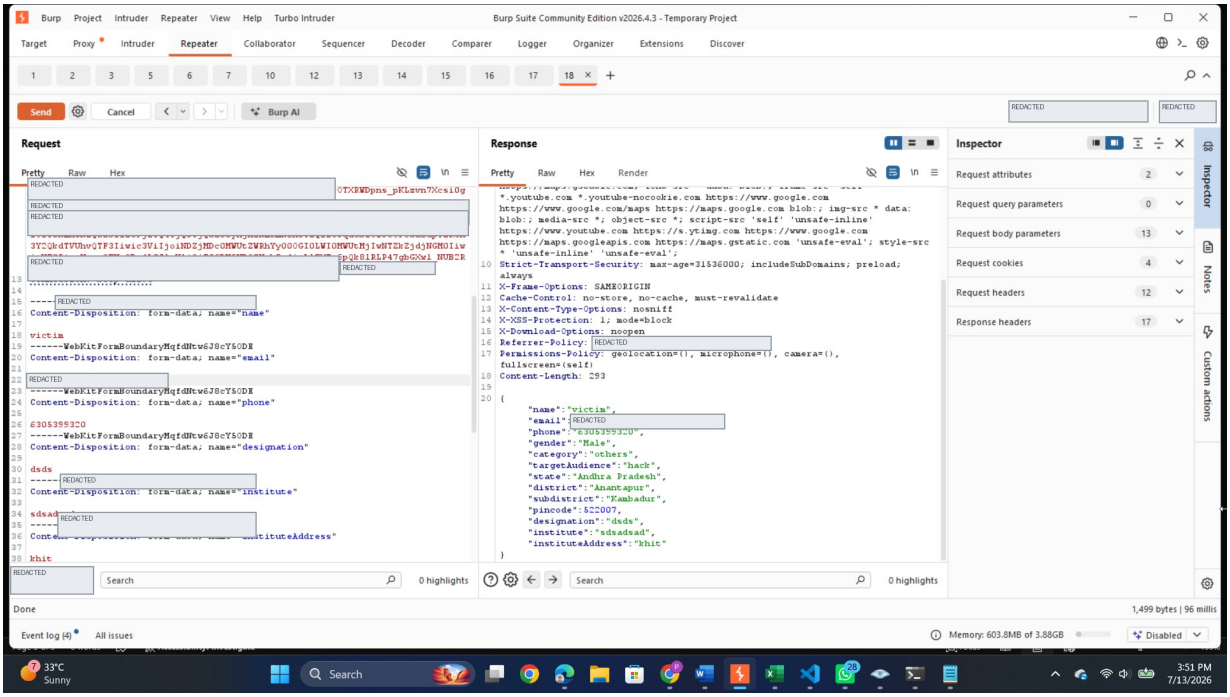
Original testing screenshot — sensitive details masked.



Original testing screenshot — sensitive details masked.



Original testing screenshot — sensitive details masked.



Original testing screenshot — sensitive details masked.

AUTHZ-002 — IDOR / Broken Object Level Authorization — Access to Other Participants' Certificates

Severity: High

What I found

I tested the certificate download request and noticed that the certificate number was included in the URL. I sent the request to Burp Repeater and changed only that number. The server returned certificates belonging to other participants. It checked that I was logged in, but it did not check whether the requested certificate belonged to my account.

How I tested it

1. I logged in with a normal participant account and clicked the certificate download option.

2. I intercepted the request in Burp Suite and sent it to Repeater.

3. I changed the certificate number in the request while keeping my authenticated session.

4. The server returned a certificate for a different participant. I repeated the test with other certificate numbers and received additional certificates.

5. I also tested a number that did not exist. The server returned a not-found response, showing that it checked whether a certificate number existed but did not enforce ownership.

Why it matters

A logged-in participant could access certificates belonging to other people. Those certificates contained personal information such as participant names, institution details and dates. Sequential certificate numbers also made it easier to test other records.

Why it happened

The server checked whether the requester was authenticated, but it did not verify that the requested certificate belonged to that requester. The certificate identifiers were also sequential.

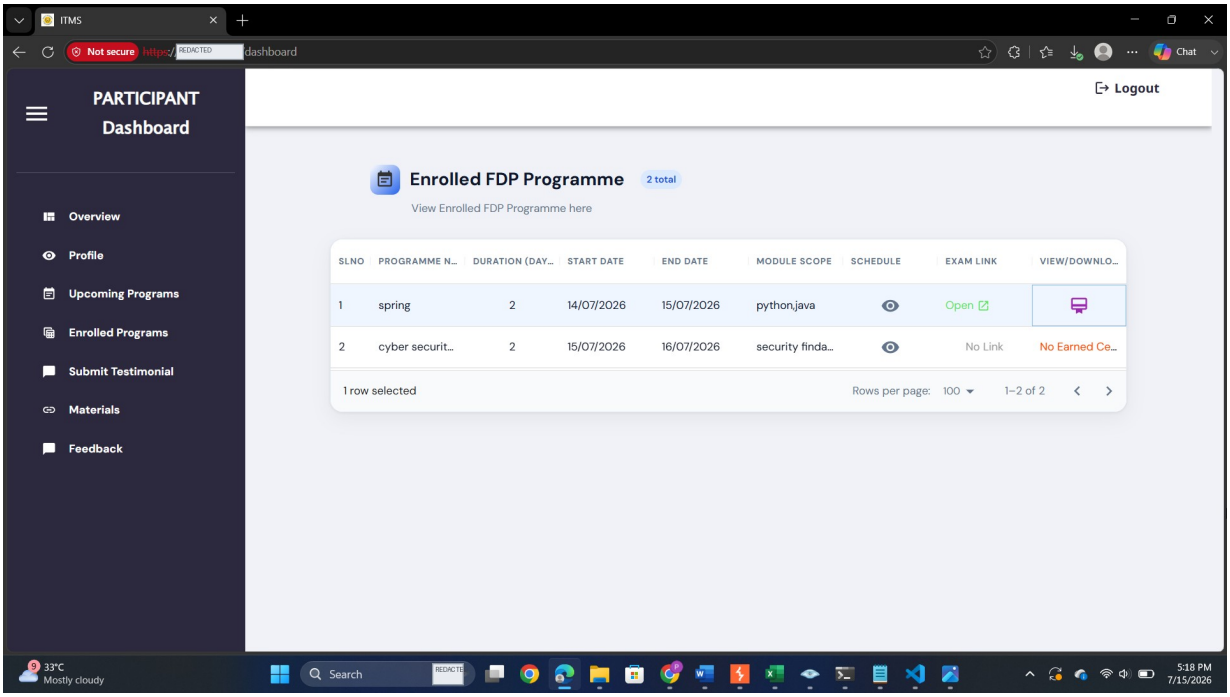
Recommended fix

Before returning a certificate, check ownership on the server. If the certificate does not belong to the authenticated user, deny the request. Random identifiers can make guessing harder, but they do not replace the ownership check.

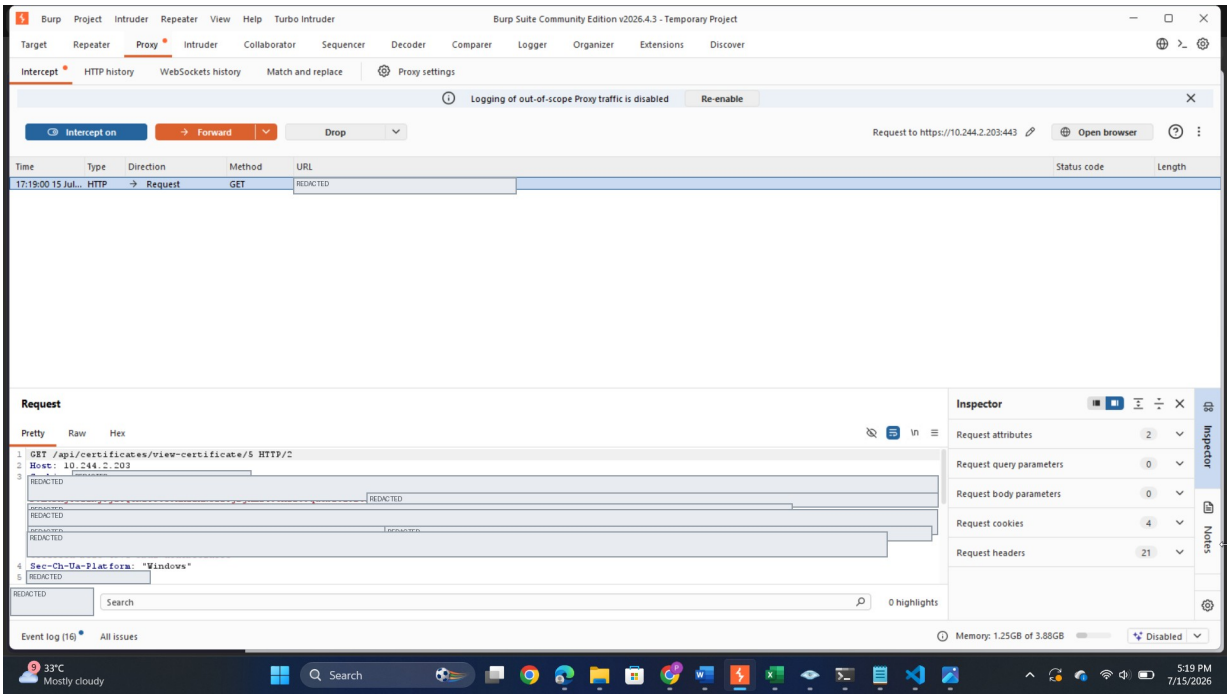
References

OWASP API Security Top 10 2023 — API1: Broken Object Level Authorization; CWE-639; WSTG-ATHZ-01

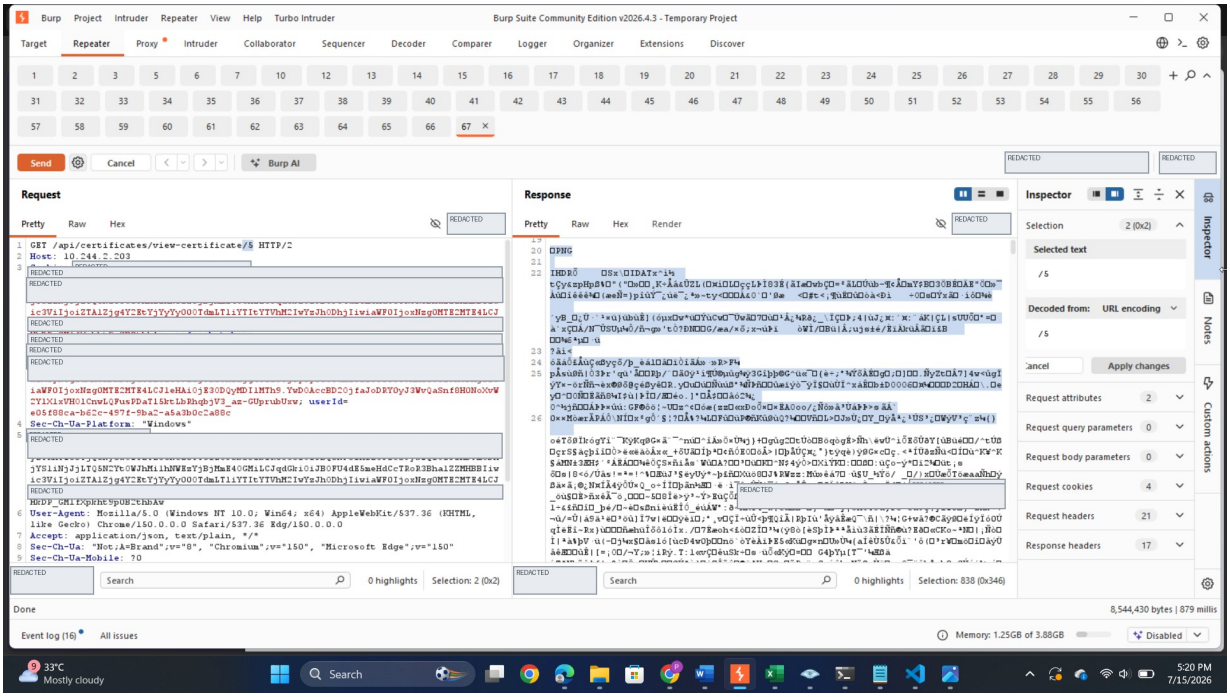
Evidence screenshots



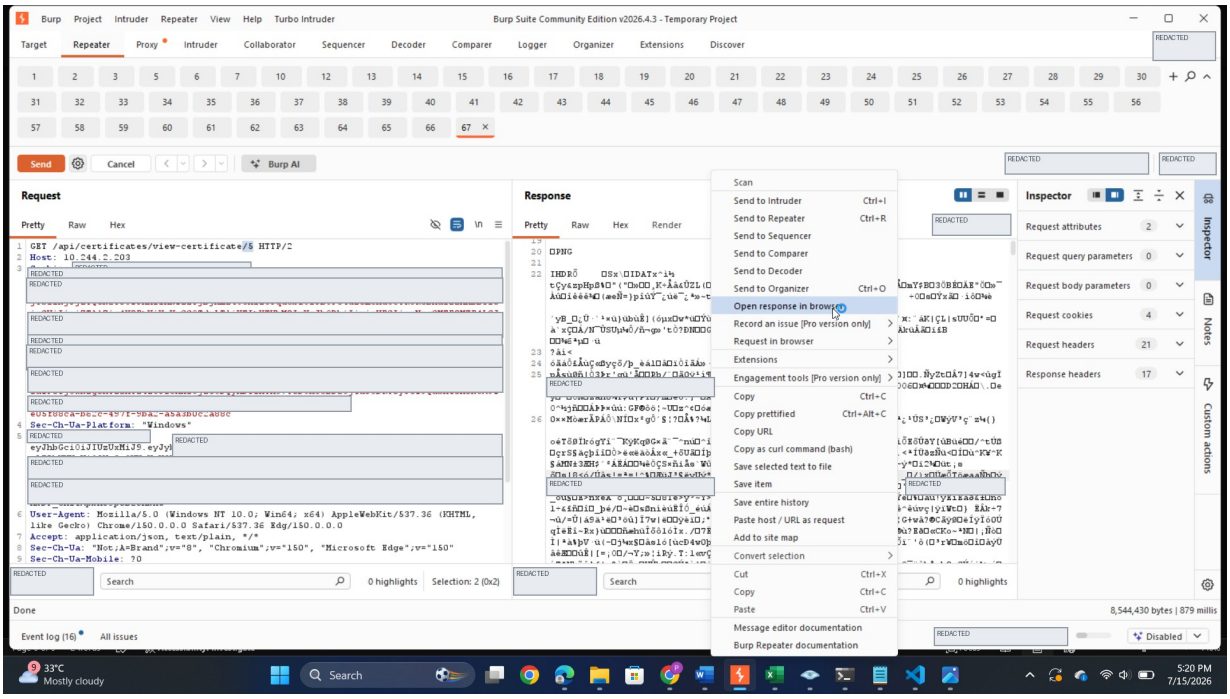
Original testing screenshot — sensitive details masked.



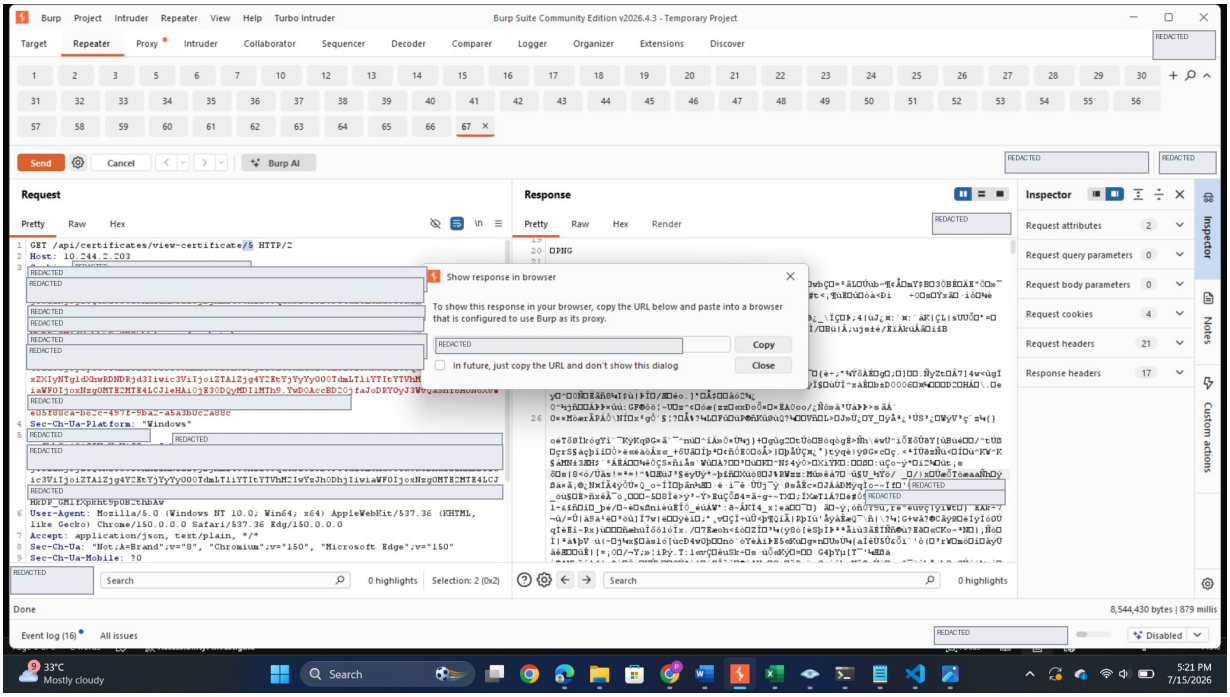
Original testing screenshot — sensitive details masked.



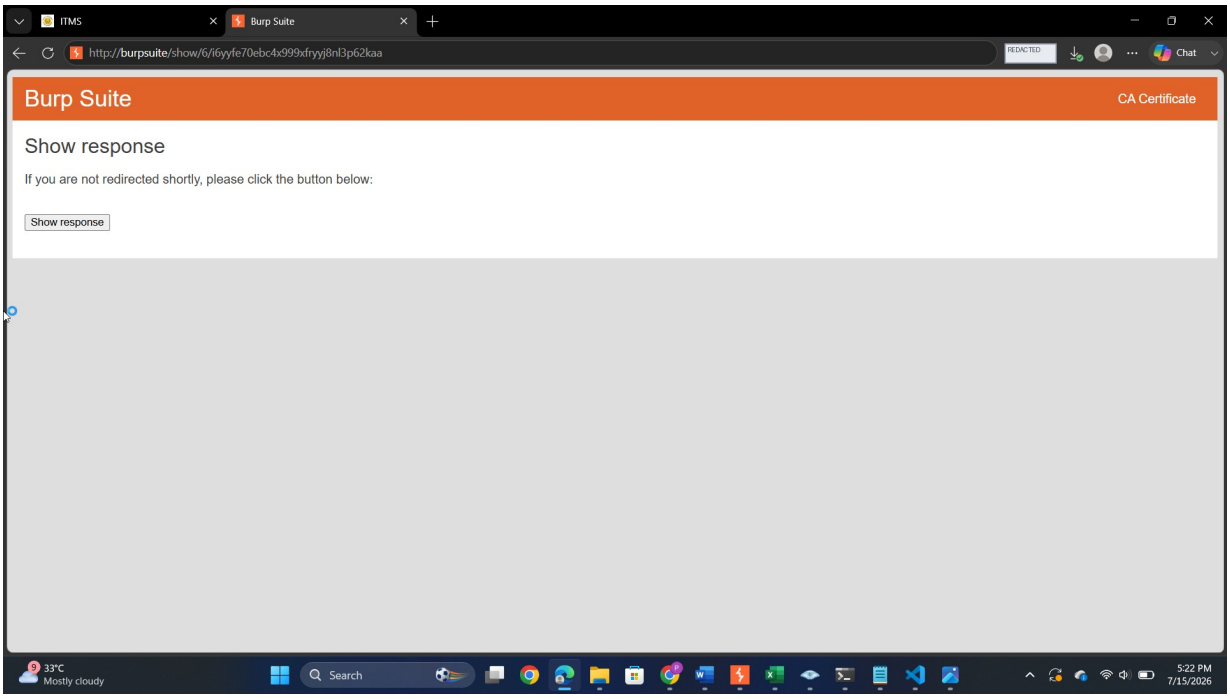
Original testing screenshot — sensitive details masked.



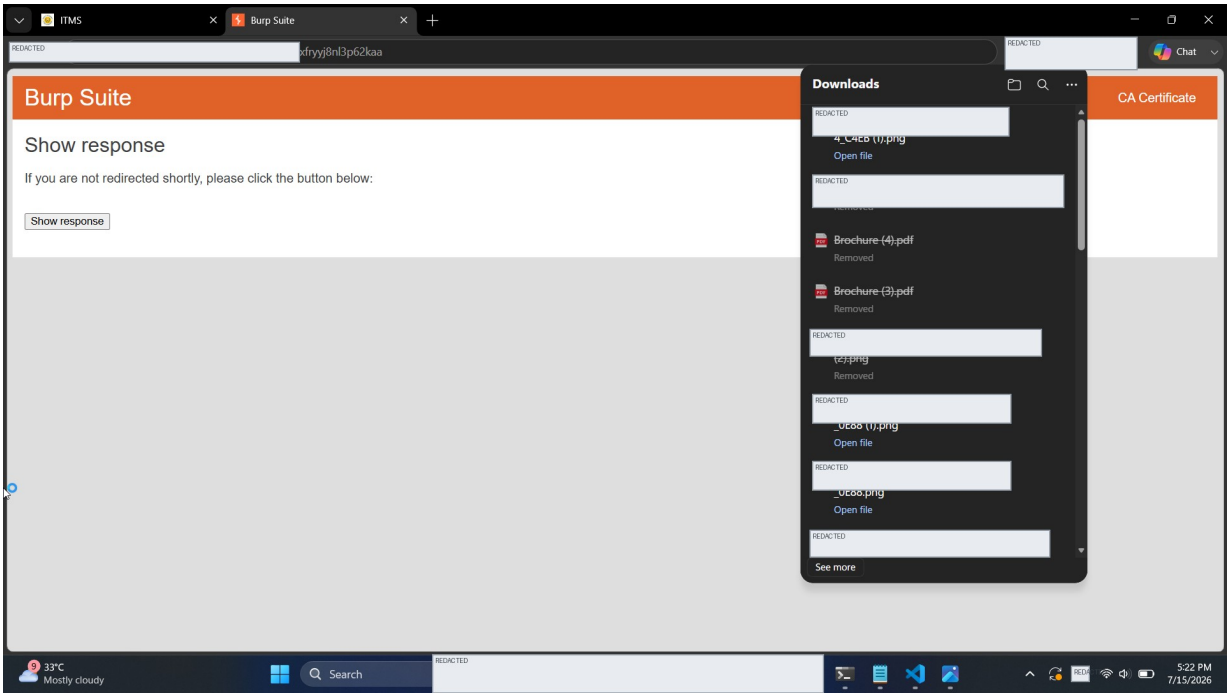
Original testing screenshot — sensitive details masked.



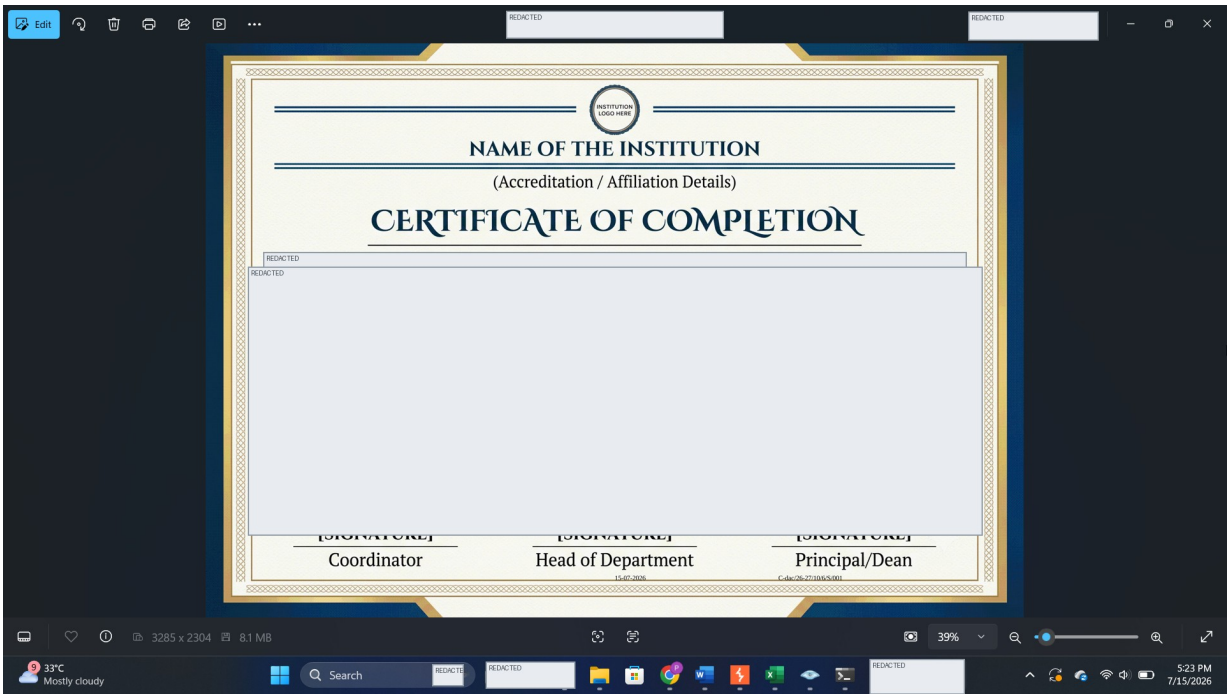
Original testing screenshot — sensitive details masked.



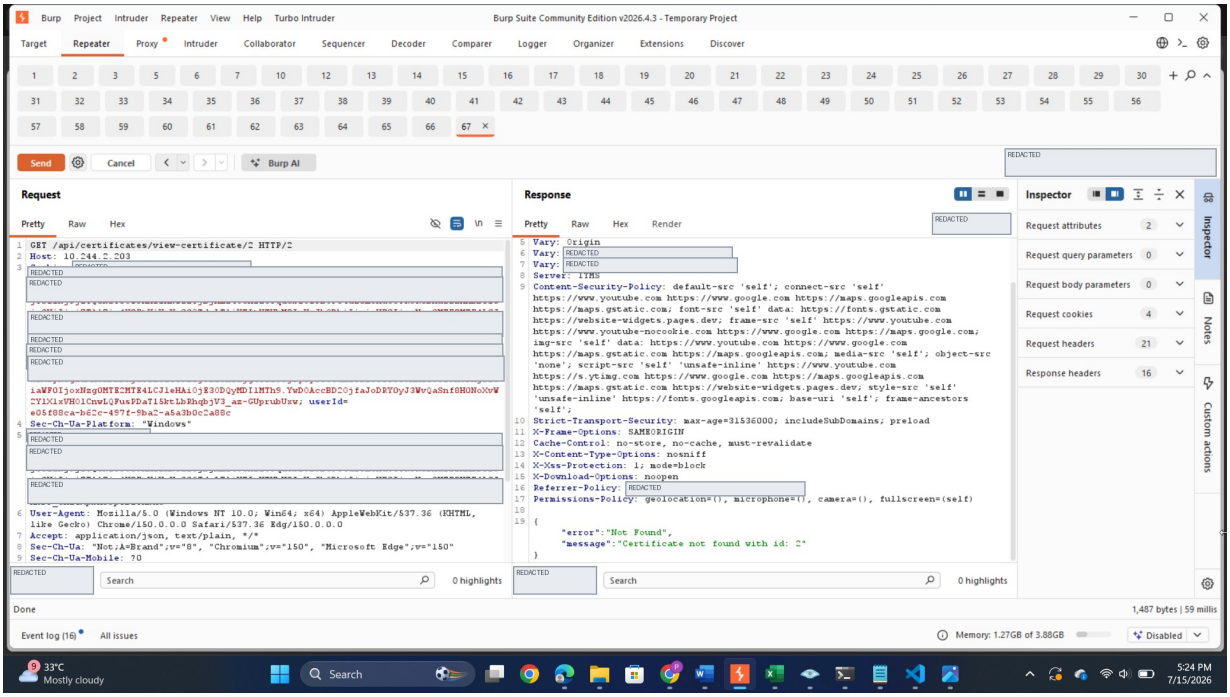
Original testing screenshot — sensitive details masked.



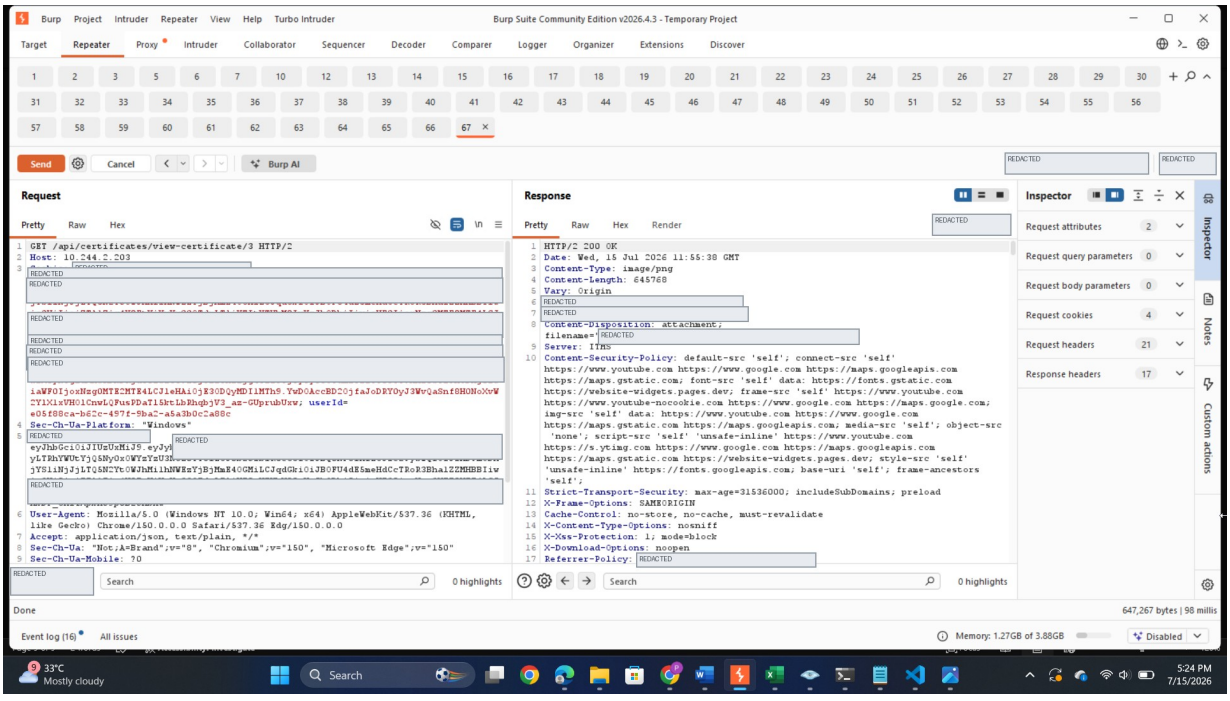
Original testing screenshot — sensitive details masked.



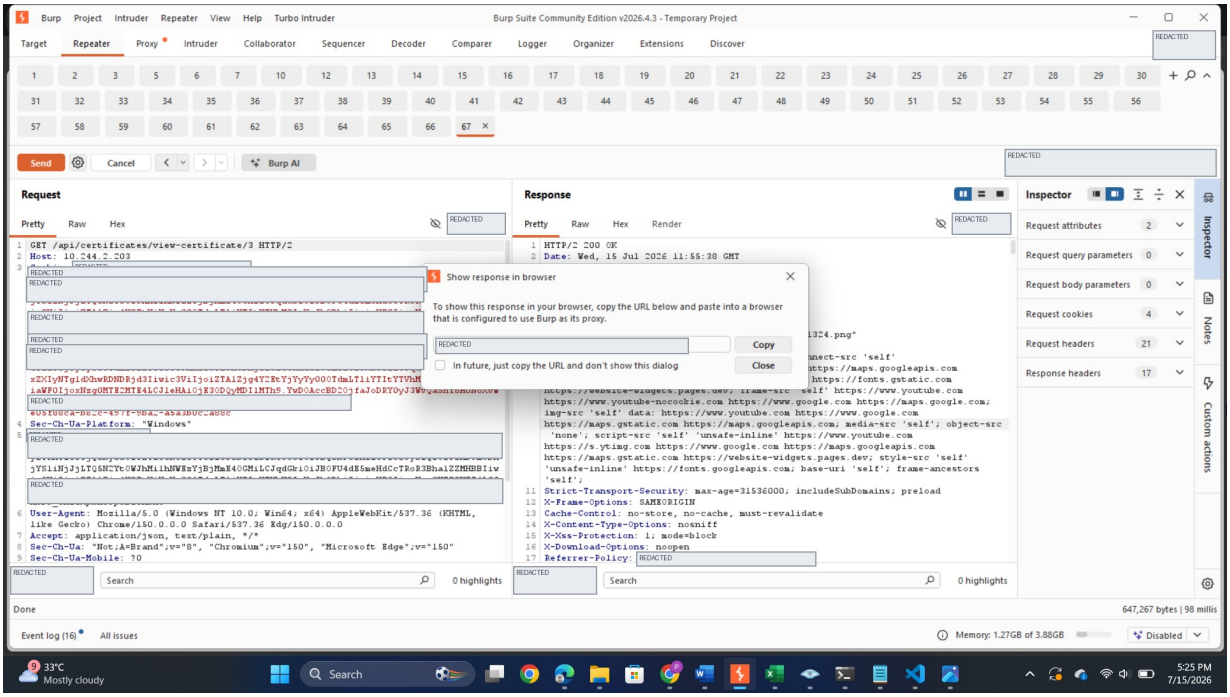
Original testing screenshot — sensitive details masked.



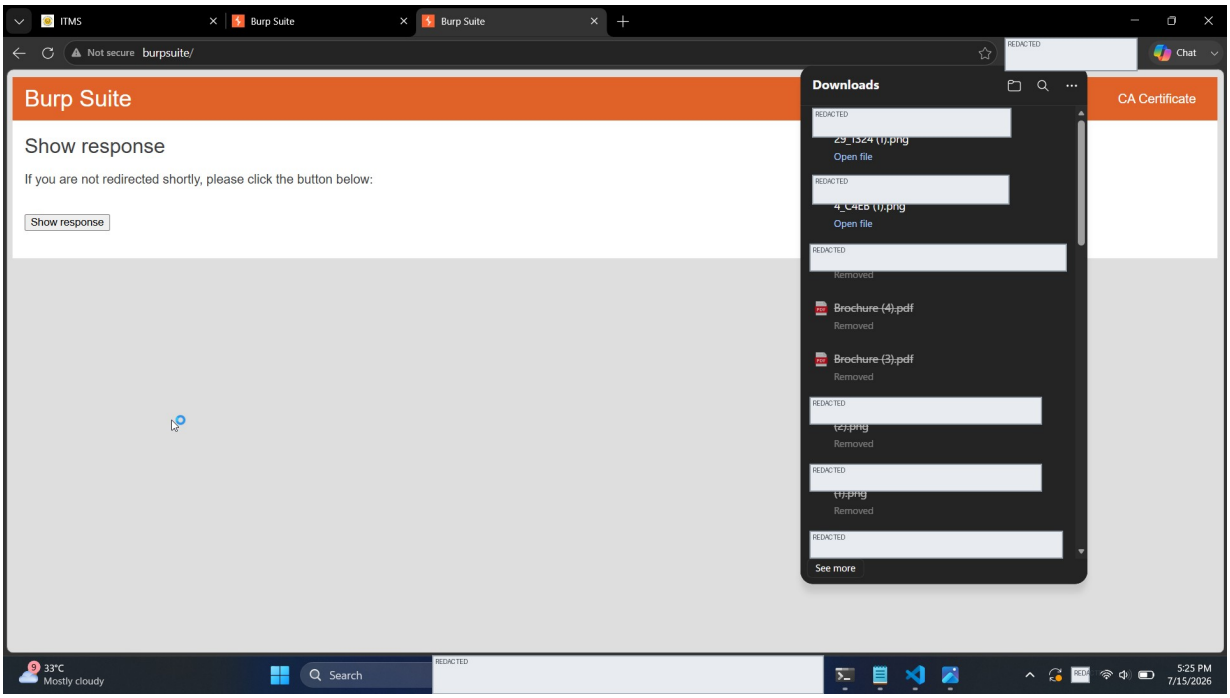
Original testing screenshot — sensitive details masked.



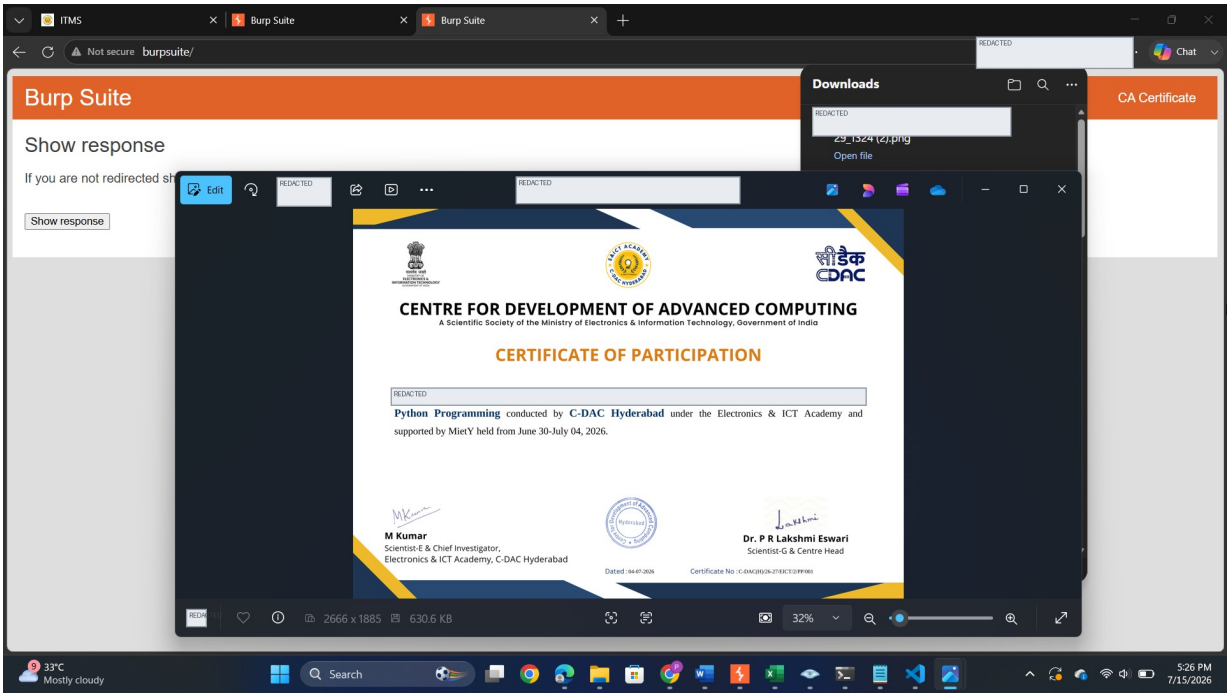
Original testing screenshot — sensitive details masked.



Original testing screenshot — sensitive details masked.



Original testing screenshot — sensitive details masked.



Original testing screenshot — sensitive details masked.